Guia simples - Laboratorio CI/CD

Execute os passos na ordem. Os blocos indicam o conteudo completo dos arquivos novos. Nas adaptacoes Python, substitua apenas os trechos indicados. Este guia parte do projeto original restaurado. Nunca envie seu .env ao GitHub.

1. Preparar dependencias

Criar requirements.txt

```
langchain>=1.0,<2
langchain-google-genai>=4,<5
langchain-ollama>=1,<2
google-genai>=1,<2
psycopg2-binary>=2.9,<3
python-dotenv>=1,<2
pydantic>=2,<3
```

Criar requirements-dev.txt

```
-r requirements.txt
pytest>=8,<10
reportlab>=4,<5
PyYAML>=6,<7
python -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements-dev.txt
```

2. Adaptar o agente para o servidor

main.py: aplicar as substituicoes abaixo. Linhas com - saem, linhas com + entram; linhas sem sinal sao contexto. Os sinais e cabecalhos nao fazem parte do codigo.

```
--- main.py
+++ main.py
@@ -10,14 +10,15 @@
 load_dotenv()
 llm_gemini = ChatGoogleGenerativeAI(
-    model="gemini-3.5-flash-lite",
+    model=os.getenv("GEMINI_MODEL", "gemini-3.5-flash-lite"),
     temperature=0.7,
     top_p=0.95,
     api_key=os.getenv("GEMINI_API_KEY"),
 )
 llm_codigo = ChatOllama(
-    model="qwen2.5-coder:3b",
+    model=os.getenv("OLLAMA_MODEL", "qwen2.5-coder:3b"),
+    base_url=os.getenv("OLLAMA_BASE_URL", "http://localhost:11434"),
     temperature=0,
 )
@@ -54,14 +55,26 @@
     from watcher import rodar_watcher
     rodar_watcher(invocar_agente)
-thread_watcher = threading.Thread(target=iniciar_watcher, daemon=True)
-thread_watcher.start()
+def main():
+    import argparse
+    parser = argparse.ArgumentParser(description="Agente SH-PSCBS")
+    parser.add_argument("--watch-only", action="store_true", help="Executa o watcher sem terminal
interativo")
+    args = parser.parse_args()
+    if args.watch_only:
+        iniciar_watcher()
+        return
+    thread_watcher = threading.Thread(target=iniciar_watcher, daemon=True)
+    thread_watcher.start()
+    while True:
+        try:
+            user_input = input("> ")
+        except EOFError:
+            break
+        if user_input.lower() in ("sair", "end", "fim", "tchau", "bye"):
+            print("Encerrando a conversa...")
+            break
+        invocar_agente(user_input)
-while True:
```

```
-    user_input = input("> ")
-    if user_input.lower() in ("sair", "end", "fim", "tchau", "bye"):
-        print("Encerrando a conversa...")
-        break
-
-    invocar_agente(user_input)+if __name__ == "__main__":
+    main()
```

watcher.py: aplicar as substituicoes abaixo. Linhas com - saem, linhas com + entram; linhas sem sinal sao contexto. Os sinais e cabecalhos nao fazem parte do codigo.

```
--- watcher.py
+++ watcher.py
@@ -1,10 +1,14 @@
+import os
 import time
 from snapshot import verificar_mudancas
 import dotenv
-DB_URL    = dotenv.get_key(dotenv.find_dotenv(), "DB_URL")
-PASTA     = r"C:\Users\iagodiniz-ieg\Downloads\sh_test"
-INTERVALO = 5
+dotenv.load_dotenv()
+DB_URL = os.getenv("DB_URL")
+PASTA = os.getenv("PROJECT_PATH", os.getcwd())
+INTERVALO = int(os.getenv("WATCH_INTERVAL", "5"))
+if INTERVALO < 1:
+    raise ValueError("WATCH_INTERVAL deve ser maior que zero")
 def extrair_mudancas(diff: str) -> dict:
     removidas, adicionadas, tabelas_novas = [], [], []
```

snapshot.py: aplicar as substituicoes abaixo. Linhas com - saem, linhas com + entram; linhas sem sinal sao contexto. Os sinais e cabecalhos nao fazem parte do codigo.

```
--- snapshot.py
+++ snapshot.py
@@ -4,7 +4,7 @@
 from datetime import datetime, timezone
-SNAPSHOTS_DIR = "schema_snapshots"
+SNAPSHOTS_DIR = os.getenv("SNAPSHOTS_DIR", "schema_snapshots")
 def ler_schema(url):
```

3. Criar testes e arquivos Docker

Criar/substituir tests/test_schema.py (crie as pastas quando necessario).

```
import json
from unittest.mock import MagicMock, patch
import snapshot
from watcher import extrair_mudancas
def test_diff_detecta_tabelas_e_colunas():
    antigo = {"clientes": {"id": "integer", "apelido": "text"}, "legado": {}}
    novo = {"clientes": {"id": "integer", "email": "text"}, "pedidos": {}}
    linhas = set(snapshot.comparar(antigo, novo).splitlines())
    assert linhas == {
        "[+] Tabela adicionada: pedidos", "[-] Tabela removida: legado",
        "[+] clientes.email adicionada (text)", "[-] clientes.apelido removida (text)",
    }
    assert snapshot.comparar(novo, novo) == "Nenhuma mudança detectada."
def test_extracao_para_orquestrador():
    diff = "[-] clientes.apelido removida (text)\n[+] clientes.email adicionada (text)\n[+] Tabela
adicionada: pedidos"
    assert extrair_mudancas(diff) == {
        "removidas": ["apelido"], "adicionadas": ["email"],
        "tabelas": ["clientes"], "tabelas_novas": ["pedidos"],
    }
def test_snapshot_persiste_sem_duplicar(tmp_path, monkeypatch):
    monkeypatch.setattr(snapshot, "SNAPSHOTS_DIR", str(tmp_path))
    assert snapshot.carregar_ultimo_snapshot() is None
    schema = {"clientes": {"id": "integer"}}
    caminho = snapshot.salvar_snapshot(schema)
    assert json.loads(tmp_path.joinpath("schema_latest.json").read_text()) == schema
    assert caminho is not None
    assert snapshot.salvar_snapshot(schema) is None
def test_fluxo_primeira_leitura_e_mudanca(tmp_path, monkeypatch):
    monkeypatch.setattr(snapshot, "SNAPSHOTS_DIR", str(tmp_path))
    with patch.object(snapshot, "ler_schema", return_value={"clientes": {}}):
        assert "Primeiro snapshot" in snapshot.verificar_mudancas("url-ficticia")
        assert snapshot.verificar_mudancas("url-ficticia") == "Nenhuma mudança detectada."
    with patch.object(snapshot, "ler_schema", return_value={"clientes": {"email": "text"}}):
        assert snapshot.verificar_mudancas("url-ficticia") == "[+] clientes.email adicionada (text)"
def test_leitura_agrupa_colunas_e_fecha_conexao():
```

```
    conn = MagicMock()
    conn.cursor.return_value.fetchall.return_value = [
        ("clientes", "id", "integer"), ("clientes", "nome", "text"),
    ]
    with patch.object(snapshot.psycopg2, "connect", return_value=conn):
        assert snapshot.ler_schema("url-ficticia") == {"clientes": {"id": "integer", "nome":
"text"}}
    conn.cursor.return_value.close.assert_called_once()
    conn.close.assert_called_once()
```

Criar/substituir Dockerfile (crie as pastas quando necessario).

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home --uid 10001 agent && mkdir /data && chown agent:agent /data
COPY main.py prompts.py snapshot.py tools.py watcher.py ./
USER agent
ENV PROJECT_PATH=/workspace SNAPSHOTS_DIR=/data/schema_snapshots
CMD ["python", "main.py", "--watch-only"]
```

Criar/substituir .dockerignore (crie as pastas quando necessario).

```
.git
.env
.env.*
.venv
__pycache__
.pytest_cache
.firecrawl
schema_snapshots
backups
docs
tests
```

Criar/substituir compose.yaml (crie as pastas quando necessario).

```
services:
  agent:
    image: ${APP_IMAGE:?Defina APP_IMAGE com o digest publicado pela pipeline}
    restart: unless-stopped
    init: true
    env_file: .env
    environment:
      PROJECT_PATH: /workspace
      SNAPSHOTS_DIR: /data/schema_snapshots
    volumes:
      - ${TARGET_PROJECT_PATH:?Defina TARGET_PROJECT_PATH}:/workspace
      - snapshots:/data
volumes:
  snapshots:
```

Criar/substituir scripts/deploy.sh (crie as pastas quando necessario).

```
#!/usr/bin/env bash
set -euo pipefail
cd /opt/sh-pscbs
: "${APP_IMAGE:?Imagem obrigatoria}"
export APP_IMAGE
docker compose config --quiet
docker compose pull
docker compose up -d --wait --wait-timeout 60
# O agente nao oferece HTTP: verificar que o processo permanece vivo.
sleep 15
container=$(docker compose ps -q agent)
test -n "$container"
test "$(docker inspect -f '{{.State.Running}}' "$container")" = true
test "$(docker inspect -f '{{.RestartCount}}' "$container")" = 0
printf 'APP_IMAGE=%s\n' "$APP_IMAGE" > .deployed-image
echo "Deployment validado: $APP_IMAGE"
```

Criar/substituir .env.example (crie as pastas quando necessario).

```
GEMINI_API_KEY=""
DB_URL=""
PROJECT_PATH="/caminho/do/projeto/monitorado"
WATCH_INTERVAL="5"
OLLAMA_BASE_URL="http://localhost:11434"
OLLAMA_MODEL="qwen2.5-coder:3b"
GEMINI_MODEL="gemini-3.5-flash-lite"
# No servidor Docker, usar caminhos absolutos do host:
TARGET_PROJECT_PATH="/srv/projeto-monitorado"
```

Acrescente ao .gitignore:

```
.venv/
__pycache__/
.env
.pytest_cache/
test-results.xml
backups/
```

4. Validar localmente

```
.\.venv\Scripts\python -m pip check
.\.venv\Scripts\python -m pytest -v --junitxml=test-results.xml
docker build -t sh-pscbs:test .
docker run --rm -e GEMINI_API_KEY=chave-ficticia-validacao -e PYTHON_DOTENV_DISABLED=1 sh-pscbs:test
 python main.py --help
```

Resultado esperado: cinco testes passando, imagem construida e ajuda do agente exibida. O XML e gerado pelo pytest; nao escreva resultados manualmente. Docker Desktop deve estar em execucao.

5. Preparar servidor e GitHub

Preparação do deployment

Este projeto é um agente de terminal/background, sem servidor HTTP. O destino é

um servidor Linux acessível via SSH, com Docker Engine e Compose v2. O agente

publicado fica acessível ao operador por `docker compose logs` e atua no projeto

montado em `/workspace`. GitHub Pages não executaria este processo Python.

Preparação única do servidor

1. Criar `/opt/sh-pscbs` e conceder escrita ao usuário SSH do deployment.

2. Criar nesse diretório um `.env` baseado em `.env.example`, com credenciais reais.

Não enviar esse arquivo ao Git. Usar `chmod 600 .env`.

3. Configurar `TARGET_PROJECT_PATH` com o caminho absoluto do projeto monitorado.

O UID 10001 do container precisa ler e escrever nesse diretório e em backups.

4. Configurar `DB_URL` e `OLLAMA_BASE_URL` com endereços alcançáveis de dentro do

container. `localhost` no container aponta para ele próprio. Disponibilizar

PostgreSQL e Ollama na rede do servidor e carregar o modelo Qwen configurado.

Confirmar que `GEMINI_MODEL` está disponível para a conta da API.

5. Se a imagem GHCR for privada, executar no servidor `docker login ghcr.io`

usando uma credencial com `read:packages` e acesso ao pacote. A autenticação

do runner GitHub não autentica o servidor remoto.

6. O usuário SSH precisa executar Docker e acessar somente o projeto autorizado.

GitHub

No environment `production`, configurar os secrets:

| Secret | Conteúdo |

|---|---|

| DEPLOY_HOST | Host ou IP do servidor |

| DEPLOY_USER | Usuário SSH |

| DEPLOY_SSH_KEY | Chave privada de deployment |

| DEPLOY_KNOWN_HOSTS | Chave pública SSH do host, conferida por canal confiável |

Para atender ao deployment automático do laboratório, não habilitar aprovação

manual obrigatória no environment. Habilitar Actions e permissão de escrita de

pacotes. A branch esperada é `main`.

Fluxo e evidências

Abrir um PR para `main`: somente CI deve executar. Após o merge/push, CI executa

novamente; se passar, CD publica no GHCR e atualiza o container remoto por digest

imutável. Um PR nunca recebe acesso ao job de produção.

Guardar a URL do run com CI e CD verdes, o SHA do commit, o digest no GHCR e a

saída de `docker compose logs --tail=50 agent` e `docker compose ps` no servidor.

O script aguarda o serviço e verifica processo vivo e ausência de reinícios.

Isso confirma inicialização, mas não substitui um teste funcional: alterar uma

coluna em um banco de teste e observar o diff e a ação no projeto de teste.

Não usar um banco de produção para essa demonstração.

Para administrar depois do deployment, exportar a referência salva:

```bash

cd /opt/sh-pscbs

export APP_IMAGE=$(cut -d= -f2- .deployed-image)

docker compose ps

docker compose logs --tail=50 agent

```

Para rollback, definir `APP_IMAGE` com um digest anterior do GHCR e executar

`docker compose up -d`. Código alterado pelo agente deve ser restaurado a partir

dos backups quando necessário; reverter o container não reverte esses arquivos.

Validação local

```powershell

python -m venv .venv
```

.\.venv\Scripts\python -m pip install -r requirements-dev.txt

.\.venv\Scripts\python -m pip check

.\.venv\Scripts\python -m pytest -v --junitxml=test-results.xml

.\.venv\Scripts\python -m compileall -q main.py prompts.py snapshot.py tools.py watcher.py

docker build -t sh-pscbs:test .

.\.venv\Scripts\python scripts/gerar_relatorio.py

```

No servidor Ubuntu, instale Docker Engine e o plugin Compose v2 conforme a documentacao oficial da sua distribuicao. Use uma maquina/VM propria ou um VPS Linux; a contratacao e o custo ficam a seu criterio. O deployment nao funciona somente com o repositorio, sem um ambiente de execucao.

```
sudo mkdir -p /opt/sh-pscbs
sudo chown "$USER:$USER" /opt/sh-pscbs
ssh-keygen -t ed25519 -f ~/.ssh/lab_deploy
```

Gere a chave em uma maquina confiavel. Adicione o conteudo de lab_deploy.pub ao ~/.ssh/authorized_keys do usuario do servidor (pasta 700 e arquivo 600). Cadastre a chave privada em DEPLOY_SSH_KEY. Confirme a chave publica do servidor por canal confiavel antes de preencher DEPLOY_KNOWN_HOSTS. Nao publique chaves privadas. Garanta que o usuario tenha acesso ao Docker e escrita em /opt/sh-pscbs. O projeto montado deve permitir escrita ao UID 10001.

## 6. Criar a pipeline

Criar .github/workflows/cicd.yml:

```
name: CI e CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
permissions:
  contents: read
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: pip
      - run: python -m pip install -r requirements-dev.txt
      - run: python -m pip check
      - run: python -m compileall -q main.py prompts.py snapshot.py tools.py watcher.py
      - run: python -m pytest -v --junitxml=test-results.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: resultados-testes
          path: test-results.xml
      - name: Validar construcao Docker
        run: docker build -t sh-pscbs:test .
      - name: Validar inicializacao do agente no container
        run: >-
          docker run --rm
          -e GEMINI_API_KEY=chave-ficticia-validacao
          -e PYTHON_DOTENV_DISABLED=1
          -e LANGSMITH_TRACING=false
          sh-pscbs:test python main.py --help
  cd:
    needs: ci
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    concurrency:
```
```

```
    group: deploy-production
    cancel-in-progress: false
  permissions:
    contents: read
    packages: write
  steps:
    - uses: actions/checkout@v4
    - name: Nome da imagem
      id: image
      run: echo "name=ghcr.io/${GITHUB_REPOSITORY,,}" >> "$GITHUB_OUTPUT"
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    - uses: docker/build-push-action@v6
      id: build
      with:
        context: .
        push: true
        tags: ${{ steps.image.outputs.name }}:${{ github.sha }}
    - name: Configurar SSH
      env:
        SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
        KNOWN_HOSTS: ${{ secrets.DEPLOY_KNOWN_HOSTS }}
      run: |
        install -m 700 -d ~/.ssh
        printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
        chmod 600 ~/.ssh/id_ed25519
        printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
    - name: Publicar e verificar servico
      env:
        DEPLOY_HOST: ${{ secrets.DEPLOY_HOST }}
        DEPLOY_USER: ${{ secrets.DEPLOY_USER }}
        APP_IMAGE: ${{ steps.image.outputs.name }}@${{ steps.build.outputs.digest }}
      run: |
        scp compose.yaml "$DEPLOY_USER@$DEPLOY_HOST:/opt/sh-pscbs/compose.yaml"
        ssh "$DEPLOY_USER@$DEPLOY_HOST" "APP_IMAGE='$APP_IMAGE' bash -s" < scripts/deploy.sh
```

7. Demonstrar CI e CD

```
git switch -c lab-cicd
git add .
git status
git commit -m "Configura pipeline CI/CD"
git push -u origin lab-cicd
```

Antes do commit, confirme que .env e chaves nao aparecem no git status. Abra um pull request para main no GitHub: a CI deve passar e o CD nao deve executar. Faca merge: o push na main executa CI e depois CD. Se falhar, abra o log da etapa, corrija e envie novo commit. No servidor, confira imagem, processo e logs. Use banco e projeto descartaveis para demonstrar uma mudanca de coluna, observando a deteccao e a atualizacao dos arquivos.

8. Entregar o relatorio final

Monte um unico PDF com link do repositorio, workflow completo, prompts realmente utilizados, problemas e correcoes, alteracoes feitas com IA e por voce, breve analise e evidencias. Inclua URL e SHA do run, CI do PR, CI/CD do push, digest GHCR e logs do servidor. Informe quantos prompts usou. Este guia nao substitui evidencias de execucao. So considere o laboratorio completo depois que o deployment e a demonstracao funcional passarem.